

RL Programming Assignment 2

Pranay Marumamula (23B1073)

October 2025

Task 1 - MDP Planning

The MDP is read from the input file and stored as transition and reward matrices. For each state-action pair, a set of reachable next states is stored to speed up computation. The code supports an optional `--policy` argument. If provided, the program evaluates the given policy using the Bellman equations. Otherwise, it computes an **optimal policy** using either LP or HPI.

Policy Evaluation

The Bellman equations are formulated as an LP problem to be given to the inbuilt solver of pulp, PULP_CBC_CMD. The value of terminal states is fixed to 0, and they are not a part of the LP problem.

Linear Programming Formulation

Terminal states are forced to have a value of 0. For other states, the constraints are set up appropriately. The data structure "reachable" helps in reducing the complexity of setting up these constraints from $S \cdot A \cdot S$ to $S \cdot A \cdot degree$, which works very well for sparse transitions (where most states have a zero probability).

The optimal policy is calculated by computing the Q value for every (s, a) pair, and setting the optimal policy of state s as $\arg\max_a Q(s, a)$. Here as well, the data structure "reachable" can be used to speed up computation.

Howard's Policy Iteration

The algorithm starts by choosing the policy of taking action 0 for every state s . Every policy is evaluated by the policy evaluator, and then improved by a function that picks the action with the highest value of $Q^\pi(s, a)$ where π is the current policy. This iterative process is terminated once the policy cannot be improved anymore.

Looping between policies because of floating point error, is avoided by setting up a simple threshold bound for improvement.

Task 2 - Strategic Card Play

- **Compact State Representation:** Each state is represented as a sorted tuple of integers corresponding to the cards currently in hand. Both suits are treated equivalently, while formulating the MDP. Based on the suit in the hand, the appropriate action is chosen. This allows us to reduce the number of swapping actions, as well as the state space. Sorting ensures that the order of cards in the hand does not matter. This too prunes the search space vastly.
- **Threshold-based Pruning:** States whose summed value of cards exceeds or equals the `threshold` are never expanded. This acts as an early termination condition and drastically limits branching for large thresholds.
- **State numbering:** Since the MDP planner expects states to be numbered from 0 to $S - 1$, the algorithm performs an explicit DFS to number the valid states reachable from the start state (empty hand). This is done using a `stack`, rather than recursion to avoid exceeding recursion depth limits. Once the states are numbered, these numbers can be used to print out the transitions in the required format, to stdout. This is also done using an explicit DFS. This code to number states can be reused in `decoder.py` to figure out the state corresponding to each hand.